WORKTUAL VIBE PLATFORM

Vibe Coding Tool Architecture

Production workflow for AI coding assistants like Claude Code, Cursor, Codex, Windsurf, and Aider.

Backend Authority

Identity, policy, tool permissions, validation, writes, and audit remain server-side.

MAS Runtime

Specialist agents coordinate through typed handoffs instead of one monolithic prompt.

Safe Commit Gate

Generated changes land only after build, test, preview, QA, and approval policy pass.

Presentation PDF | June 12, 2026

WHAT THE PLATFORM MUST SOLVE

Problem Analysis

- Users expect natural-language coding updates, but production systems must protect the repository, runtime, credentials, and local machine.
- The assistant must understand project state before editing: repository graph, selected project, local folder binding, current files, prior memory, and active failure state.
- Slow or failed updates usually come from weak orchestration: missing budgets, unresolved dependencies, no repair routing, unclear file ownership, and unsafe write timing.
- The target behavior is not only code generation. It is plan, execute, validate, repair, preview, persist, explain, and allow human intervention.

Architecture Principle

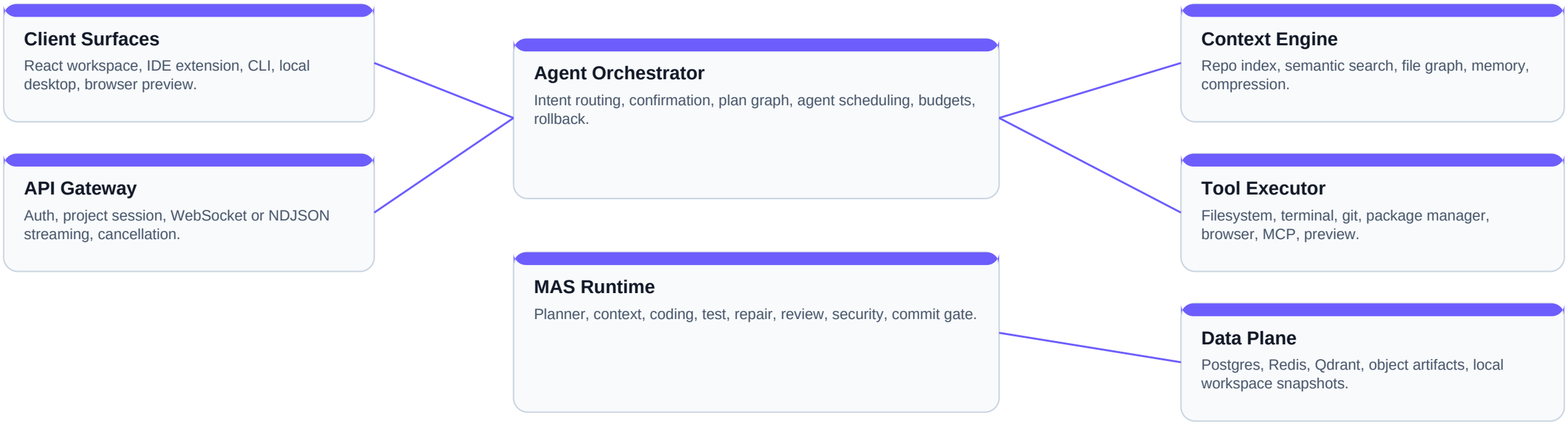
The model proposes. The runtime verifies. The backend commits. The user approves high-risk work. This boundary is the difference between a demo and a production coding assistant.

Success Criteria

A run is successful only when generated or edited files pass validation gates, the preview builds, errors are normalized, events are persisted, and the UI receives a deterministic completion shape.

SYSTEM BOUNDARIES

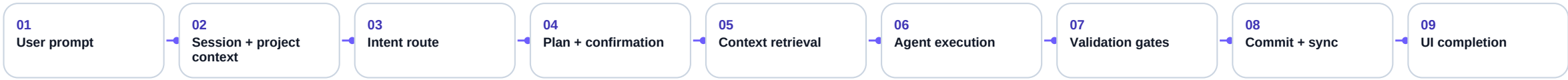
Reference Architecture



Boundary rule: clients render state and request actions; backend runtime owns tools, policy, validation, persistence, and final writes.

PROMPT TO PERSISTED RESULT

End-To-End Request Flow



Stage	Runtime Responsibility	Failure Case	Expected Recovery
Route	Classify greeting, question, new generation, scoped update, repair, or confirmation.	Wrong route sends small chat into heavy model flow.	Greeting/project-summary fast path, route tests, confidence threshold.
Plan	Create bounded tasks, file scope, tool permissions, and runtime budget.	Broad update times out during patch call.	Split task, exact component targeting, budget-aware checkpoints.
Execute	Run specialist agents and tools against a staged workspace.	Generated imports or code do not build.	Dependency preflight, repair agent, known shim policy.
Validate	Build, test, preview, visual QA, diff review, and policy checks.	Runtime preview crashes after build.	Browser console capture, component smoke tests, rollback.
Commit	Write backend files, sync selected local folder, persist memory/events.	Generated files exist but local folder not updated.	Explicit folder binding, write receipt, local sync audit.

CLEAN ARCHITECTURE RESPONSIBILITIES

Component Breakdown

Component	Owns	Does Not Own	Key Contracts
API Gateway	Auth, session, project routes, streaming protocol, cancellation, request normalization.	Prompt reasoning or file mutation policy.	POST generate-stream, progress/error/complete events.
Orchestrator	Intent, run graph, confirmation state, agent order, budget, rollback intent.	Raw shell execution or direct filesystem writes.	Run plan, action graph, agent handoff.
Context Manager	Repo map, selected files, memories, embeddings, compression, token strategy.	Committing edits.	Context packet with provenance and limits.
LLM Gateway	Provider routing, retries, model policy, structured outputs, cost and latency tracking.	Tool authority.	Model request, response schema, usage metrics.
Tool Executor	Permissioned tools, filesystem, git, terminal, browser, preview, package manager.	Planning decisions.	Tool call, result, policy verdict, audit event.
Commit Gate	Validation result, staged preview, write-back, local sync, version and memory persistence.	Ignoring failed checks.	Commit receipt, rollback reason, diff summary.

SPECIALIST AGENTS, NOT ONE GIANT PROMPT

MAS Runtime Architecture

Planner
Turns user intent into ordered tasks, constraints, risks, and acceptance criteria.

Context Agent
Builds a scoped context packet from repository, selected project, memory, and failure state.

Code Agent
Produces patch proposals against approved files and architecture boundaries.

Test Agent
Selects and runs focused tests, build checks, and static validation.

Repair Agent
Consumes structured failures and creates minimal repair attempts within budget.

Reviewer
Checks diff risk, API compatibility, regressions, and missing validation.

Security Agent
Scans secrets, prompt injection, path abuse, commands, and dependency risks.

Commit Gate
Commits only when validation, policy, preview, and human-approval rules pass.

MAS rule: every agent emits a typed result with objective, evidence, files touched, risk, confidence, and next action. The orchestrator can replay or resume the run from these durable handoffs.

HOW AGENTS COMMUNICATE

A2A Handoff Contract

Field	Meaning	Example	Why It Matters
source_agent / target_agent	The sender and intended next owner.	planner -> context_agent	Prevents ambiguous ownership and hidden work.
objective	Bounded work to perform next.	Update OnboardingWizard only	Reduces broad edits and timeout risk.
evidence	Files, logs, errors, tests, or decisions supporting the handoff.	Build failed: unresolved import	Makes repair grounded, not speculative.
artifacts	Patch, plan, test output, preview URL, screenshots, or structured validation result.	pending diff + preview ID	Enables replay and audit.
risk + approval	Policy risk and whether user confirmation is required.	High: terminal install	Keeps destructive actions human-controlled.
confidence + next_action	Agent confidence and a machine-readable action recommendation.	0.72, run repair	Supports deterministic orchestration.

WHAT ENTERS THE MODEL

Context Engineering

Retrieved Context

Selected files, import graph, symbols, package manifests, previous run memory, current error logs, user brief, and local-folder binding.

Compressed Context

Repository summaries, API contracts, component responsibilities, known failure signatures, and relevant test history.

Excluded Context

Secrets, unrelated runtime folders, node_modules, generated caches, broad logs, and stale memory without provenance.

Strategy	Implementation	Production Benefit
Repo indexing	File tree, imports, exports, symbols, dependency manifest, test map, and route map.	Lets updates target exact components instead of rewriting the app.
Semantic retrieval	Qdrant embeddings over files, docs, memories, and previous successful fixes.	Finds relevant context even when user names are imprecise.
Token budgeting	Reserve budget for plan, patch, repair, validation summary, and final explanation.	Prevents repair skip because runtime budget is exhausted.
Context provenance	Every included snippet records file path, source, freshness, and why selected.	Improves debugging and reduces stale or injected context.

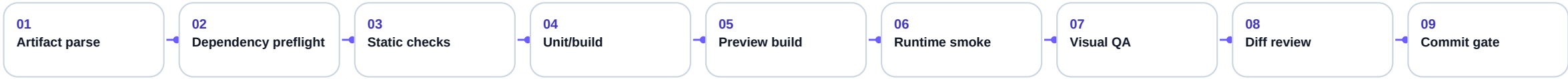
BACKEND-OWNED AUTHORITY

Tool Execution And Permissions

Permission Tier	Examples	Policy	Audit Requirement
Read-only	List files, read project files, inspect logs, search symbols.	Allowed inside project boundary.	Record files read and context provenance.
Staged mutation	Patch pending workspace, format generated files, run dependency preflight.	Allowed only in staged copy.	Record diff and changed files.
Runtime execution	Build, test, preview, browser console capture, package manager checks.	Constrained command allowlist and timeout.	Record command, exit code, stdout/stderr summary.
Write-back	Commit backend files, sync user-selected local folder, save versions.	Allowed only after commit gate passes.	Record write receipt and rollback metadata.
Human approval	Destructive shell, broad deletion, network installs, secrets, production deploy.	Pause and ask before execution.	Record approver, scope, and decision.

NO BROKEN CODE SHOULD BE WRITTEN

Validation And Commit Gates



- Validation must run against a staged workspace before touching the active backend project or user-selected local folder.
- Unresolved imports, missing packages, syntax errors, preview crashes, and console exceptions should route to repair with exact evidence.
- A generated file set is not complete until preview build, browser smoke, local sync, memory persistence, and UI completion all produce receipts.
- If any gate fails, preserve the existing website and return a normalized error with category, code, last runtime step, elapsed time, and next action.

TURNING FAILURES INTO NEXT ACTIONS

Failure Handling And Repair Policy

Failure	Likely Cause	Repair Agent Input	Resolution Policy
Scoped update timeout	Patch call too broad, budget too low, unclear target component.	File scope, task size, elapsed seconds, last model step.	Split task or restrict exact component; never silently retry forever.
Staged Vite build failed	Generated unresolved import, package mismatch, syntax issue.	Full Vite log, manifest, generated diff, dependency policy.	Repair dependency or code; commit only after rebuild.
Preview runtime crash	Build passed but browser console/runtime state failed.	Console stack, route, screenshot, component tree hints.	Run browser smoke repair before write-back.
Local write-back missing	Project not bound to selected folder or sync failed.	Folder binding, commit receipt, file list, sync error.	Return explicit sync failure and preserve staged artifacts.
Greeting routed to generation	Intent classifier too aggressive.	Prompt, project state, route confidence.	Conversation/project-summary fast path; no Gemini artifact call or file write.

DURABLE SYSTEM MEMORY

Storage And Data Model

PostgreSQL

Projects, files, runs, events, messages, versions, tool calls, approvals, audit logs.

Redis

Live run state, cancellation, locks, WebSocket fanout, queue coordination.

Qdrant

Repository embeddings, semantic memory, prior fixes, documentation retrieval.

Artifact Store

Patch bundles, generated previews, screenshots, logs, snapshots, rollback material.

Entity	Why It Exists	Operational Requirement
GenerationRun	Single user request lifecycle.	Status, timestamps, route, model usage, error category, final receipt.
AgentRun / Handoff	MAS trace and replay record.	Source/target agent, objective, evidence, artifacts, risk, confidence.
ToolCall	Tool audit and debugging.	Input summary, permission tier, result, duration, stdout/stderr digest.
FileVersion	Rollback and review.	Before/after hash, diff, writer, validation receipt.
MemoryItem	Project-specific learning.	Scope, source run, freshness, confidence, invalidation policy.

PRODUCTION OPERATION MODEL

Scaling And Observability

- Use WebSocket or Redis Pub/Sub for live run progress; keep HTTP endpoints for project CRUD and deterministic replay.
- Use distributed workers for long generation, repair, preview, browser QA, indexing, and embedding jobs.
- Use per-tenant and per-project locks to prevent concurrent writes from corrupting a workspace.
- Emit OpenTelemetry-style spans for route, plan, context retrieval, model call, tool call, validation, preview, commit, and sync.
- Track SLOs: time to first progress, model latency, build duration, repair success rate, preview failure rate, local sync failures, and rollback count.

Multi-Tenant Controls

Tenant isolation, rate limits, model quotas, encrypted secrets, workspace sandboxing, artifact retention, and audit exports are required before broad rollout.

Operational Dashboard

Show live run graph, stuck steps, failed gates, model usage, repair attempts, preview logs, local sync receipts, and top recurring failure signatures.

PHASED DELIVERY

Implementation Roadmap

Phase	Goal	Deliverables	Exit Criteria
1	Stabilize current flow.	Run locks, cancel endpoint, dependency preflight, error normalization, repair thresholds, full test/build pass.	No stuck concurrent runs; actionable errors; baseline tests pass.
2	Formalize MAS contracts.	Agent contracts, A2A handoffs, commit-gate ownership, runtime summaries.	Every action has source agent, evidence, result, and policy status.
3	Improve context and repair.	Repo index, targeted component retrieval, failure classifier, repair playbooks, browser console smoke.	Scoped updates complete faster and preview failures repair reliably.
4	Production scale.	Queue workers, Redis Pub/Sub, Qdrant, observability dashboard, tenant quotas, artifact retention.	Multi-user workload with traceable, isolated runs.

Recommendation: do not add more prompt complexity until the MAS contracts, validation receipts, and repair routing are observable end to end.

WHAT MUST BE TRUE BEFORE LAUNCH

Acceptance Checklist

- Greeting or small project-summary prompts never trigger file writes.
- Every generation/update has a durable run ID and streamed progress events.
- Every model/tool action has timeout, budget, retry, and cancellation behavior.
- Every failed run returns category, code, last step, evidence, and next action.
- Generated files are committed only after staged build, preview, QA, and policy pass.
- Local-folder write-back uses only the user's selected folder binding.
- Broad or destructive actions pause for human approval.
- Agent handoffs are typed, replayable, and visible in debugging tools.
- Tests cover routing, streaming shapes, validation gates, repair, and local sync.
- Telemetry identifies stuck steps, repeated failure signatures, and model latency.

Final Architecture Position

A production vibe coding tool is a controlled runtime with AI planning inside it. Treat prompts as one input to a larger verified workflow, not as the workflow itself.